IOTC Assignment Report

Suresh Kumar
25MSDFR031

Introduction

The objective of the assignment was to create a practical Internet of Things (IOT) project that may combine **embedded systems, IoT, mobile application development and automation**. Also, the idea may be expandable with additional features in the future. Accordingly, a smart pet feeder was thought that would dispense feed at regular intervals, or through a mobile app command, or manually with future extensions like feed scheduling, notifications, camera monitoring, and food level detection.

Project Title

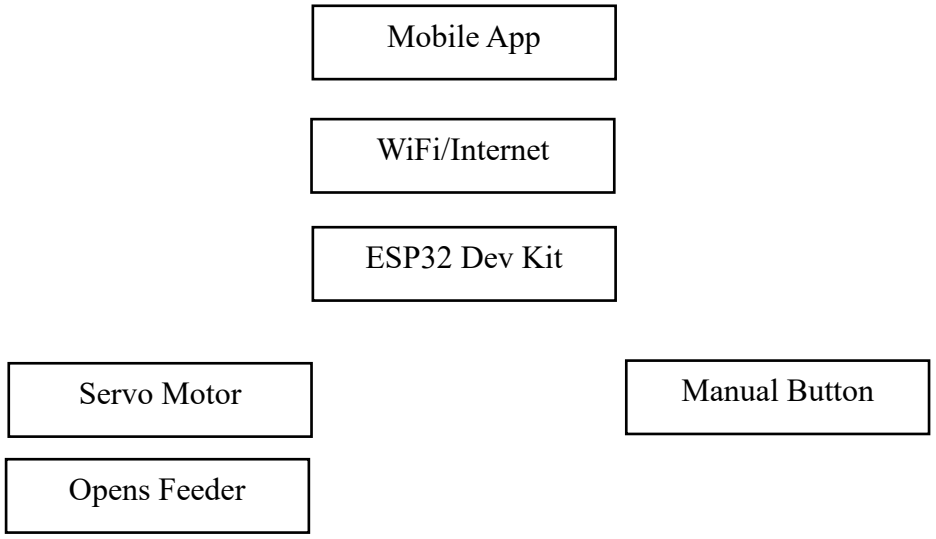
Smart Pet Feeder using IoT with Mobile App Control and Scheduled Feeding.

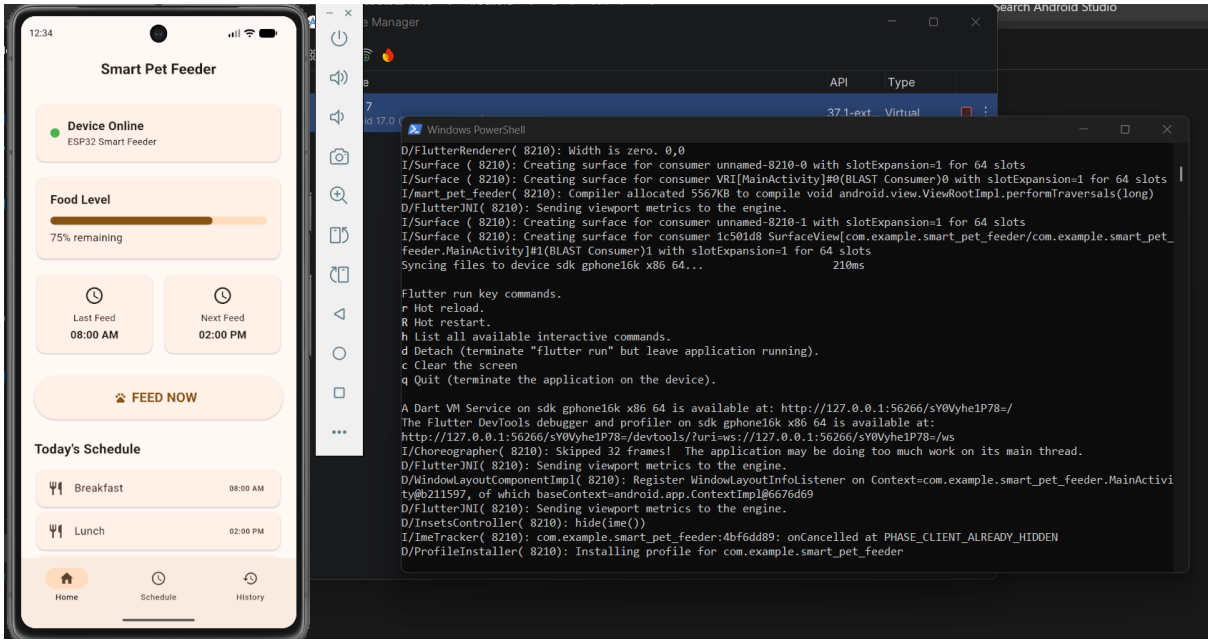
Aim

To develop an IoT-enabled automatic pet feeder that can:

- Dispense food automatically at scheduled times.
- Allow the owner to feed the pet remotely using a mobile app.
- Support manual feeding using a physical button.
- Notify the owner after successful feeding.
- Keep logs of feeding history.

Architecture of the System





Hardware Components

Component	Quantity
ESP32 Development Board	1
Servo Motor MG996R	1
5V 2A Power Adapter	1
Breadboard	1
Jumper Wires (all types)	1 pack each
Push Button	1
LED + Resistor	2
Buzzer	1
Food Container	1
Feeding Bowl	1
Servo Mount	1

Proposed Hardware Connections of ESP32 Pins

ESP32 Pin	Connected To
GPIO18	Servo Signal
GPIO23	Push Button
GPIO2	Status LED
GPIO4	Buzzer
5V	Servo VCC
GND	Common Ground

Proposed Working of the Feeder

(a) Automatic Schedule

Every day the feeder shall drop feed in the bowl at the following times:

8:00 AM

2:00 PM

8:00 PM

(b) The working principle of the automated schedule is as follows:
- ESP32 checks the time.
- At scheduled time the Servo rotates 90°
- Food gate opens and food drops into bowl
- Servo closes
- Notification sent

(c) Manual override of automated system
- Owner opens mobile app and Presses '**Feed Now'**
- Firebase updates command
- ESP32 receives command and Servo rotates
- Food dispenses
- Command resets

(d) Physical Button
- Owner pushes the physical button
- ESP32 activates servo
- Dispense food with LED blinking and Buzzer beep

The Mobile Appl will have the following Features:

Home Screen

Pet Feeder

Status: Online

Food Level: 75%

Last Fed: 8:00 AM

Next Feed: 2:00 PM

- [Feed Now]
- [Schedule]
- [History]
- [Settings]

History Screen

Today

8:00 AM Automatic

2:01 PM Manual

8:00 PM Physical

Example of Notification that owner receives on mobile when connected to same WiFi.

Food Dispensed Successfully
Pet Fed at 8:00 PM
Food Container Running Low

Software Stack

Hardware

- ESP32
- Arduino IDE
- C++

IoT

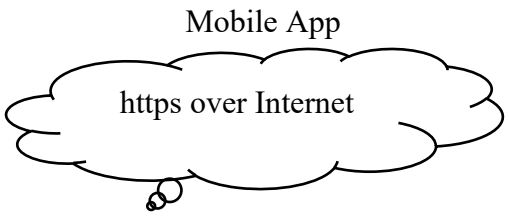
- Firebase Realtime Database (easy for beginners)
- MQTT (more scalable)

Mobile App development using Flutter.

Protocols for communication between App, ESP and hardware.

For your Smart Pet Feeder IoT project, we use several communication protocols depending upon how the ESP32 is connected, mobile app, and cloud.

The protocol architecture of the project is as follows:



Firebase

Wi-Fi

ESP32

GPIO / PWM

Servo Motor (MG996R)

Feed Mechanism

Protocols/interfaces involved

Layer / Purpose	**Protocol / Interface**	**Used for**
ESP32 ↔ Router	Wi-Fi (802.11)	Wireless Internet connectivity
Mobile App ↔ Firebase	HTTPS	Secure communication with cloud
ESP32 ↔ Firebase	HTTPS or Firebase protocol	Sending/receiving feeding commands and status
ESP32 ↔ Servo	PWM	Controlling MG996R position
ESP32 ↔ Manual button	GPIO	Detecting manual feeding
App ↔ Cloud	REST/HTTPS or Firebase SDK	Feed Now, schedules, status
Cloud ↔ ESP32	Realtime synchronization	Delivering commands to feeder

Wi-Fi - The ESP32 connects to your home Wi-Fi. This gives the feeder Internet connectivity.

HTTPS - The mobile application communicates securely with the cloud using HTTPS. This is important because commands such as Feed Now should not be sent as unencrypted HTTP traffic.

Firebase Realtime Database - This acts as the communication layer between the mobile application and ESP32.

PWM - PWM isn't an Internet protocol; it's the control signal/interface used by the ESP32 to command the MG996R servo.

GPIO - The manual push button can be connected to an ESP32 GPIO pin:

The protocol stack may be described as follows:

Application Layer
- Firebase
- HTTPS

Network Layer
- Wi-Fi (802.11)

Device Layer
- GPIO
- PWM

MQTT (Message Queuing Telemetry Transport) is a lightweight communication protocol designed specifically for IoT devices and the MQTT broker is the middleman which may be used in the enhanced version of the Smart feeder.

Three important MQTT concepts

1. Publisher

The device/application that sends a message.

2. Subscriber
   The device that receives/listens for a message.

3. Topic
   A topic is like an address where messages are published.

MQTT has the following benefits in this project

- Lightweight — good for ESP32
- Fast — messages are small
- Low bandwidth — doesn't require much data
- Bidirectional — ESP32 can send information back
- Publish/subscribe based — devices don't need to communicate directly
- Suitable for unreliable networks — MQTT has mechanisms for delivery reliability

This project has been done using online Firebase Database and the structure is as follows:

petFeeder

status online

feedNow false

schedule breakfast 08:00 lunch 14:00 dinner 20:00

history time mode

foodLevel in approximate percentage

Final Working model

1. Start the ESP32 based feeder
2. Connect WiFi
3. Connect Firebase
4. Loop
5. Read Schedule
6. Match with Time now
7. If True
   Rotate Servo
   Wait 2 seconds
   Close Servo
   Update History
   Send Notification
8. Check Feed Button?
   Feed if pressed
9. Check Mobile Command?
   Feed if received
10. Repeat

Future Enhancements

- Camera to monitor the pet while feeding.
- Voice control with Google Assistant or Alexa.
- Pet identification using RFID tags.
- AI-based feeding recommendations based on pet weight and age.
- Automatic refill alerts.
- Battery backup with charging circuit.
- Cloud analytics showing feeding trends.
- Multi-pet profiles with different feeding schedules.